

«DEVELOPMENT OF A CODE GENERATOR»

Problem Statement

1. Develop a Code Generator (CG) program for a subset of the SIGNAL programming language according to the variant.
2. The code generator program must provide:
 - reading the parse tree and tables generated by the syntax analyzer developed in the course project;
 - detecting semantic errors;
 - generating code and/or constructing internal tables for code generation.
3. Assemble a complete compiler consisting of the previously developed lexical and syntax analyzers and the code generator, which provides the following:
 - code generation and/or construction of internal tables for code generation;
 - generating a listing of the input program with messages regarding lexical, syntactic, and semantic errors.

The work is performed according to **Variant 12**.

Grammar according to the variant:

Variant 12

1. `<signal-program> --> <program>`
2. `<program> --> PROCEDURE <procedure-identifier>
 <parameters-list> ; <block> ;`
3. `<block> --> <declarations> BEGIN <statements-list>
 END`
4. `<declarations> --> <label-declarations>`
5. `<label-declarations> --> LABEL <unsigned-integer>
 <labels-list>; |
 <empty>`
6. `<labels-list> --> , <unsigned-integer> <labels-list>
 |
 <empty>`
7. `<parameters-list> --> (<variable-identifier>
 <identifiers-list>) |
 <empty>`
8. `<identifiers-list> --> , <variable-identifier>
 <identifiers-list> |
 <empty>`
9. `<statements-list> --> <statement> <statements-list>
 |
 <empty>`
10. `<statement> --> <unsigned-integer> : <statement> |
 GOTO <unsigned-integer> ; |
 RETURN ; |
 ; |
 ($ <assembly-insert-file-identifier> $)`
11. `<variable-identifier> --> <identifier>`
12. `<procedure-identifier> --> <identifier>`
13. `<assembly-insert-file-identifier> --> <identifier>`
14. `<identifier> --> <letter><string>`
15. `<string> --> <letter><string> |
 <digit><string> |
 <empty>`
16. `<unsigned-integer> --> <digit><digits-string>`
17. `<digits-string> --> <digit><digits-string> |
 <empty>`
18. `<digit> --> 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9`
19. `<letter> --> A | B | C | D | ... | Z`

Test cases:

The output of the Code Generator is written to the *output.asm* file. If semantic errors are detected during the first pass (semantic analysis), code generation is aborted, the assembly file is not created, and all relevant error messages are printed directly to the console.

To verify the correct processing of assembly insert constructs (`$ <assembly-insert-file-identifier> $`), additional test files were created with the following content:

Content of the CL file:

```
xor eax, eax
```

Content of the CMATH file:

```
add eax, ebx
```

Content of the INSERT file:

```
mov eax, 1  
add ebx, eax
```

For readability and ease of verification in this report, the translator's outputs are explicitly divided into sequential sections: **Code Generator (terminal):** or **Code Generator (output.asm):**, **Token stream:**, and **Parse tree:**.

Note on test cases: In the examples provided in this document, the Code Generator's output is placed at the very beginning before the parse tree and token list. This reordering is done strictly for reporting purposes to allow for quick verification of the compiler's correctness. In the actual console application, the logic and execution order of the intermediate compilation stages remain unchanged.

1. True-rect1:

Input program: PROCEDURE MAINPROC (PARAMA, PARAMB, PARAMC); LABEL 1, 2, 99; BEGIN 1: GOTO 2; ; (\$ INSERT \$) 2: RETURN; 99: ; END;	Output program: Code Generator (output.asm): .DATA PARAMA DD ? PARAMB DD ? PARAMC DD ? .CODE MAINPROC PROC pop PARAMC pop PARAMB pop PARAMA L_1: jmp L_2
--	--

```

    nop
    mov eax, 1
    add ebx, eax
L_2:
    ret
L_99:
    nop
    ret
MAINPROC ENDP
END

```

Token stream:

Result Lexem List:

Row	Col	Code	Lexem
1	1	401	PROCEDURE
1	11	1001	MAINPROC
1	20	40	(
1	21	1002	PARAMA
1	27	44	,
1	29	1003	PARAMB
1	35	44	,
1	37	1004	PARAMC
1	43	41	)
1	44	59	;
2	1	404	LABEL
2	7	501	1
2	8	44	,
2	10	502	2
2	11	44	,
2	13	503	99
2	15	59	;
3	1	402	BEGIN
4	5	501	1
4	6	58	:
4	8	405	GOTO
4	13	502	2
4	14	59	;
5	5	59	;
6	5	301	(\$
6	8	1005	INSERT
6	15	302	\$)
7	5	502	2
7	6	58	:
7	8	406	RETURN
7	14	59	;
8	5	503	99
8	7	58	:
8	9	59	;
9	1	403	END
9	4	59	;

Identifiers:

Code	Lexem	Type
1001	MAINPROC	-
1002	PARAMA	-
1003	PARAMB	-
1004	PARAMC	-
1005	INSERT	-

Constants:

Code	Lexem	Type
501	1	unsigned-integer
502	2	unsigned-integer
503	99	unsigned-integer

Parse tree:

Result Parse tree:

```
--<signal-program>
|--<program>
|   |-- 401 PROCEDURE
|   |--<procedure-identifier>
|   |   |-- 1001 <identifier>
|   |--<parameters-list>
|   |   |-- 40 (
|   |   |--<variable-identifier>
|   |   |   |-- 1002 <identifier>
|   |   |--<identifiers-list>
|   |   |   |-- 44 ,
|   |   |   |--<variable-identifier>
|   |   |   |   |-- 1003 <identifier>
|   |   |   |--<identifiers-list>
|   |   |   |   |-- 44 ,
|   |   |   |   |--<variable-identifier>
|   |   |   |   |   |-- 1004 <identifier>
|   |   |   |   |--<identifiers-list>
|   |   |   |   |   |--<empty>
|   |   |-- 41 )
|   |-- 59 ;
|   |--<block>
|   |   |--<declarations>
|   |   |   |--<label-declarations>
|   |   |   |   |-- 404 LABEL
|   |   |   |   |-- 501 <unsigned-integer>
|   |   |   |--<labels-list>
|   |   |   |   |-- 44 ,
|   |   |   |   |-- 502 <unsigned-integer>
|   |   |   |--<labels-list>
|   |   |   |   |-- 44 ,
|   |   |   |   |-- 503 <unsigned-integer>
|   |   |   |--<labels-list>
|   |   |   |   |--<empty>
|   |   |-- 59 ;
|   |-- 402 BEGIN
|   |--<statements-list>
|   |   |--<statement>
|   |   |   |-- 501 <unsigned-integer>
```

```

-- 58 :
--<statement>
| -- 405 GOTO
| -- 502 <unsigned-integer>
| -- 59 ;
--<statements-list>
--<statement>
| -- 59 ;
--<statements-list>
--<statement>
| -- 301 ($
| --<assembly-insert-file-
identifrier>
| -- 1005 <identifrier>
| -- 302 $)
--<statements-list>
--<statement>
| -- 502 <unsigned-
integer>
| -- 58 :
| --<statement>
| -- 406 RETURN
| -- 59 ;
--<statements-list>
--<statement>
| -- 503 <unsigned-
integer>
| -- 58 :
| --<statement>
| -- 59 ;
--<statements-list>
| --<empty>
| -- 403 END
| -- 59 ;

```

Identifiers:

Code	Lexem	Type
1001	MAINPROC	procedure-identifrier
1002	PARAMA	variable-identifrier
1003	PARAMB	variable-identifrier
1004	PARAMC	variable-identifrier
1005	INSERT	assembly-insert-file-identifrier

Constants:

Code	Lexem	Type
501	1	unsigned-integer
502	2	unsigned-integer
503	99	unsigned-integer

The simulation ended correctly.

2. True-тест2:

Вхідна програма:

```
PROCEDURE PROCESSOR (ALPHA, BETA);  
LABEL 10, 20, 30;  
BEGIN  
    ;  
    ;  
    GOTO 20;  
  
    10: RETURN;  
  
    20: ($ CMATH $)  
    GOTO 30;  
  
    ($ CL $)  
  
    30: RETURN;  
END;
```

Output program:

Code Generator (output.asm):

```
.DATA  
    ALPHA DD ?  
    BETA DD ?  
.CODE  
PROCESSOR PROC  
    pop BETA  
    pop ALPHA  
    nop  
    nop  
    jmp L_20  
L_10:  
    ret  
L_20:  
    add eax, ebx  
    jmp L_30  
    xor eax, eax  
L_30:  
    ret  
    ret  
PROCESSOR ENDP  
END
```

Token stream:

Result Lexem List:

Row	Col	Code	Lexem
1	1	401	PROCEDURE
1	11	1001	PROCESSOR
1	21	40	(
1	22	1002	ALPHA
1	27	44	,
1	29	1003	BETA
1	33	41	)
1	34	59	;
2	1	404	LABEL
2	7	501	10
2	9	44	,
2	11	502	20
2	13	44	,
2	15	503	30
2	17	59	;
3	1	402	BEGIN
4	5	59	;
5	5	59	;
6	5	405	GOTO

6	10	502	20
6	12	59	;
8	5	501	10
8	7	58	:
8	9	406	RETURN
8	15	59	;
10	5	502	20
10	7	58	:
10	9	301	(\$
10	12	1004	CMATH
10	18	302	\$)
11	5	405	GOTO
11	10	503	30
11	12	59	;
13	5	301	(\$
13	8	1005	CL
13	11	302	\$)
15	5	503	30
15	7	58	:
15	9	406	RETURN
15	15	59	;
16	1	403	END
16	4	59	;

Identifiers:

Code	Lexem	Type
1001	PROCESSOR	-
1002	ALPHA	-
1003	BETA	-
1004	CMATH	-
1005	CL	-

Constants:

Code	Lexem	Type
501	10	unsigned-integer
502	20	unsigned-integer
503	30	unsigned-integer

Parse tree:

Result Parse tree:

```
--<signal-program>
|--<program>
|   |--<procedure>
|   |   |-- 401 PROCEDURE
|   |   |--<procedure-identifier>
|   |   |   |-- 1001 <identifier>
|   |   |--<parameters-list>
|   |   |   |-- 40 (
|   |   |   |   |--<variable-identifier>
|   |   |   |   |   |-- 1002 <identifier>
|   |   |   |   |--<identifiers-list>
|   |   |   |   |   |-- 44 ,
|   |   |   |   |--<variable-identifier>
|   |   |   |   |   |-- 1003 <identifier>
|   |   |   |   |--<identifiers-list>
```



```

|--<empty>
|-- 41 )
-- 59 ;
--<block>
|--<declarations>
|--<label-declarations>
|-- 404 LABEL
|-- 501 <unsigned-integer>
|--<labels-list>
|-- 44 ,
|-- 502 <unsigned-integer>
|--<labels-list>
|-- 44 ,
|-- 503 <unsigned-integer>
|--<labels-list>
|--<empty>
-- 59 ;
-- 402 BEGIN
--<statements-list>
--<statement>
|-- 59 ;
--<statements-list>
--<statement>
|-- 59 ;
--<statements-list>
--<statement>
|-- 405 GOTO
|-- 502 <unsigned-integer>
|-- 59 ;
--<statements-list>
--<statement>
|-- 501 <unsigned-
integer>
|-- 58 :
--<statement>
|-- 406 RETURN
|-- 59 ;
--<statements-list>
--<statement>
|-- 502 <unsigned-
integer>
|-- 58 :
--<statement>
|-- 301 ($
--<assembly-
insert-file-identifier>
|-- 1004
<identifier>
|-- 302 $)
--<statements-list>
--<statement>
|-- 405 GOTO
|-- 503 <unsigned-
integer>
|-- 59 ;

```

```

|--<statements-list>
|--<statement>
|-- 301 ($
|--<assembly-
insert-file-identifier>
|-- 1005
<identifier>
|-- 302 $)
|--<statements-
list>
|--<statement>
|-- 503
<unsigned-integer>
|-- 58 :
--
<statement>
|-- 406
RETURN
|-- 59 ;
|--<statements-
list>
|--<empty>
|-- 403 END
|-- 59 ;

```

Identifiers:

Code	Lexem	Type
1001	PROCESSOR	procedure-identifier
1002	ALPHA	variable-identifier
1003	BETA	variable-identifier
1004	CMATH	assembly-insert-file-identifier
1005	CL	assembly-insert-file-identifier

Constants:

Code	Lexem	Type
501	10	unsigned-integer
502	20	unsigned-integer
503	30	unsigned-integer

The simulation ended correctly.

3. False-тест1:

Вхідна програма:

```
PROCEDURE BADPR (BADPR, X, X);  
BEGIN  
;  
END;
```

Output program:

Code Generator (terminal):

Semantic: Error (1,18) Parameter name matches procedure name 'BADPR'
Semantic: Error (1,28) Duplicate parameter name 'X'
CodeGenerator: Error Code generation aborted due to semantic errors.

Token stream:

Result Lexem List:

Row	Col	Code	Lexem
1	1	401	PROCEDURE
1	11	1001	BADPR
1	17	40	(
1	18	1001	BADPR
1	23	44	,
1	25	1002	X
1	26	44	,
1	28	1002	X
1	29	41	)
1	30	59	;
2	1	402	BEGIN
3	5	59	;
4	1	403	END
4	4	59	;

Identifiers:

Code	Lexem	Type
1001	BADPR	-
1002	X	-

Constants:

Code	Lexem	Type
------	-------	------

Parse tree:

Result Parse tree:

```
--<signal-program>  
|--<program>  
|  |-- 401 PROCEDURE  
|  |--<procedure-identifier>  
|  |  |-- 1001 <identifier>  
|  |--<parameters-list>  
|  |  |-- 40 (  
|  |  |  |--<variable-identifier>  
|  |  |  |  |-- 1001 <identifier>  
|  |  |  |--<identifiers-list>  
|  |  |  |  |-- 44 ,  
|  |  |  |--<variable-identifier>
```

	<pre> </pre>
--	--

4. False-тест2:

Вхідна програма: PROCEDURE LABELT (X); LABEL 1, 1, 2; BEGIN 1: GOTO 3; 1: RETURN; 4: ; END;	Output program: Code Generator (terminal): Semantic: Warning (2,10) Duplicate label declaration '1' Semantic: Error (4,13) GOTO refers to undeclared label '3' Semantic: Error (5,5) Label is defined more than once '1' Semantic: Error (6,5) Label defined but not declared in LABEL section '4' CodeGenerator: Error Code generation aborted due to semantic errors. Token stream: Result Lexem List: <table><tr><th>Row</th><th>Col</th><th>Code</th><th>Lexem</th></tr><tr><td>1</td><td>1</td><td>401</td><td>PROCEDURE</td></tr><tr><td>1</td><td>11</td><td>1001</td><td>LABELT</td></tr><tr><td>1</td><td>18</td><td>40</td><td>(</td></tr><tr><td>1</td><td>19</td><td>1002</td><td>X</td></tr><tr><td>1</td><td>20</td><td>41</td><td>)</td></tr></table>	Row	Col	Code	Lexem	1	1	401	PROCEDURE	1	11	1001	LABELT	1	18	40	(	1	19	1002	X	1	20	41	)
Row	Col	Code	Lexem																						
1	1	401	PROCEDURE																						
1	11	1001	LABELT																						
1	18	40	(																						
1	19	1002	X																						
1	20	41	)																						

1	21	59	;
2	1	404	LABEL
2	7	501	1
2	8	44	,
2	10	501	1
2	11	44	,
2	13	502	2
2	14	59	;
3	1	402	BEGIN
4	5	501	1
4	6	58	:
4	8	405	GOTO
4	13	503	3
4	14	59	;
5	5	501	1
5	6	58	:
5	8	406	RETURN
5	14	59	;
6	5	504	4
6	6	58	:
6	8	59	;
7	1	403	END
7	4	59	;

Identifiers:

Code	Lexem	Type
1001	LABELT	-
1002	X	-

Constants:

Code	Lexem	Type
501	1	unsigned-integer
502	2	unsigned-integer
503	3	unsigned-integer
504	4	unsigned-integer

Parse tree:

Result Parse tree:

--<signal-program>

|--<program>

| |-- 401 PROCEDURE

| |--<procedure-identifier>

| | |-- 1001 <identifier>

| |--<parameters-list>

| | |-- 40 (

| | |--<variable-identifier>

| | | |-- 1002 <identifier>

| | |--<identifiers-list>

| | | |--<empty>

| | |-- 41)

| |-- 59 ;

| |--<block>

| | |--<declarations>

| | | |--<label-declarations>

```

-- 404 LABEL
-- 501 <unsigned-integer>
--<labels-list>
|  -- 44 ,
|  -- 501 <unsigned-integer>
|  --<labels-list>
|  |  -- 44 ,
|  |  -- 502 <unsigned-integer>
|  |  --<labels-list>
|  |  |  --empty>
|  -- 59 ;
-- 402 BEGIN
--<statements-list>
|  --<statement>
|  |  -- 501 <unsigned-integer>
|  |  -- 58 :
|  |  --<statement>
|  |  |  -- 405 GOTO
|  |  |  -- 503 <unsigned-integer>
|  |  |  -- 59 ;
|  --<statements-list>
|  |  --<statement>
|  |  |  -- 501 <unsigned-integer>
|  |  |  -- 58 :
|  |  |  --<statement>
|  |  |  |  -- 406 RETURN
|  |  |  |  -- 59 ;
|  |  --<statements-list>
|  |  |  --<statement>
|  |  |  |  -- 504 <unsigned-integer>
|  |  |  |  -- 58 :
|  |  |  |  --<statement>
|  |  |  |  |  -- 59 ;
|  |  --<statements-list>
|  |  |  --<empty>
-- 403 END
-- 59 ;

```

Identifiers:

Code	Lexem	Type
1001	LABELT	procedure-identifier
1002	X	variable-identifier

Constants:

Code	Lexem	Type
501	1	unsigned-integer
502	2	unsigned-integer
503	3	unsigned-integer
504	4	unsigned-integer

5. False-тест3:

Вхідна програма:

```
PROCEDURE GOTOTEST ();  
LABEL 5;  
BEGIN  
    GOTO 5;  
    RETURN;  
END;
```

Output program:

Code Generator (terminal):

Semantic: Error GOTO refers to label '5' that was never defined in the code
CodeGenerator: Error Code generation aborted due to semantic errors.

Token stream:

Result Lexem List:

Row	Col	Code	Lexem
1	1	401	PROCEDURE
1	11	1001	GOTOTEST
1	20	40	(
1	21	1002	X
1	22	41	)
1	23	59	;
2	1	404	LABEL
2	7	501	5
2	8	59	;
3	1	402	BEGIN
4	5	405	GOTO
4	10	501	5
4	11	59	;
5	5	406	RETURN
5	11	59	;
6	1	403	END
6	4	59	;

Identifiers:

Code	Lexem	Type
1001	GOTOTEST	-
1002	X	-

Constants:

Code	Lexem	Type
501	5	unsigned-integer

Parse tree:

Result Parse tree:

```
--<signal-program>  
|  
|--<program>  
| |  
| | |-- 401 PROCEDURE  
| | |--<procedure-identifier>  
| | | |-- 1001 <identifier>  
| | |--<parameters-list>  
| | | |-- 40 (  
| | | | |--<variable-identifier>  
| | | | | |-- 1002 <identifier>  
| | | | |--<identifiers-list>
```

	<pre> --<empty> -- 41) -- 59 ; --<block> --<declarations> --<label-declarations> -- 404 LABEL -- 501 <unsigned-integer> --<labels-list> --<empty> -- 59 ; -- 402 BEGIN --<statements-list> --<statement> -- 405 GOTO -- 501 <unsigned-integer> -- 59 ; --<statements-list> --<statement> -- 406 RETURN -- 59 ; --<statements-list> --<empty> -- 403 END -- 59 ;</pre>									
	Identifiers: <table><tr><th>Code</th><th>Lexem</th><th>Type</th></tr><tr><td>1001</td><td>GOTOTEST</td><td>procedure-identifier</td></tr><tr><td>1002</td><td>X</td><td>variable-identifier</td></tr></table>	Code	Lexem	Type	1001	GOTOTEST	procedure-identifier	1002	X	variable-identifier
Code	Lexem	Type								
1001	GOTOTEST	procedure-identifier								
1002	X	variable-identifier								
	Constants: <table><tr><th>Code</th><th>Lexem</th><th>Type</th></tr><tr><td>501</td><td>5</td><td>unsigned-integer</td></tr></table>	Code	Lexem	Type	501	5	unsigned-integer			
Code	Lexem	Type								
501	5	unsigned-integer								

6. False-тест4:

Вхідна програма: PROCEDURE FILETEST (); BEGIN (\$ NOFILE \$) END;	Output program: Code Generator (terminal): Semantic: Error (3,8) Assembly insert file not found 'NOFILE' CodeGenerator: Error Code generation aborted due to semantic errors. Token stream: Result Lexem List: <table><tr><th>Row</th><th>Col</th><th>Code</th><th>Lexem</th></tr><tr><td>1</td><td>1</td><td>401</td><td>PROCEDURE</td></tr><tr><td>1</td><td>11</td><td>1001</td><td>FILETEST</td></tr><tr><td>1</td><td>20</td><td>40</td><td>(</td></tr><tr><td>1</td><td>21</td><td>1002</td><td>X</td></tr><tr><td>1</td><td>22</td><td>41</td><td>)</td></tr><tr><td>1</td><td>23</td><td>59</td><td>;</td></tr></table>	Row	Col	Code	Lexem	1	1	401	PROCEDURE	1	11	1001	FILETEST	1	20	40	(	1	21	1002	X	1	22	41	)	1	23	59	;
Row	Col	Code	Lexem																										
1	1	401	PROCEDURE																										
1	11	1001	FILETEST																										
1	20	40	(																										
1	21	1002	X																										
1	22	41	)																										
1	23	59	;																										


```

2      1      402      BEGIN
3      5      301      ($)
3      8      1003     NOFILE
3      15     302      ($)
4      1      403      END
4      4      59       ;

```

Identifiers:

Code	Lexem	Type
1001	FILETEST	-
1002	X	-
1003	NOFILE	-

Constants:

Code	Lexem	Type
------	-------	------

Parse tree:

Result Parse tree:

```

--<signal-program>
|--<program>
| |-- 401 PROCEDURE
| |--<procedure-identifier>
| | |-- 1001 <identifier>
| |--<parameters-list>
| | |-- 40 (
| | | |--<variable-identifier>
| | | | |-- 1002 <identifier>
| | | |--<identifiers-list>
| | | | |--<empty>
| | |-- 41 )
| |-- 59 ;
| |--<block>
| | |--<declarations>
| | | |--<label-declarations>
| | | | |--<empty>
| | |-- 402 BEGIN
| | |--<statements-list>
| | | |--<statement>
| | | | |-- 301 ($)
| | | | | |--<assembly-insert-file-
| | | | | | |-- 1003 <identifier>
| | | | | | |-- 302 $)
| | | | |--<statements-list>
| | | | | |--<empty>
| | | |-- 403 END
| |-- 59 ;

```

Identifiers:

Code	Lexem	Type
1001	FILETEST	procedure-identifier
1002	X	variable-identifier
1003	NOFILE	assembly-insert-file-identifier